ENGINEERING & PRODUCT BRIEFING

HRHUB TalentFinder

An automated candidate sourcing, scoring and ranking platform

A recruiter types the requirements for a role. The system finds candidate profiles, extracts each one into structured data, scores every candidate against those requirements on four weighted dimensions, filters out the ones that fail a hard requirement, and returns a ranked shortlist with a written justification for each person — replacing hours of manual profile review with a search that runs in the background and streams its progress live.

PREPARED FOR	STATUS	CODEBASE
Discussion with the company owner	Working end-to-end · 358 tests · LinkedIn + Indeed	Branch `main` · `c9b0cb0` + uncommitted work (§16)

Contents

HOW TO USE THIS DOCUMENT

Sections 1–3 are the business story: read these if you have five minutes. Sections 4–11 are how the system is built, in the order you would explain it at a whiteboard. Sections 12–17 are what an owner cares about — cost, risk, and what happens next. Section 18 is a rehearsal: the questions likely to come at you, with answers you can give without hedging. Section 19 defines every technical term used, in plain English.

1. Executive summary

If you get two minutes with the owner, this is the shape of it:

THE PITCH

Recruiters spend most of their sourcing time reading profiles that were never going to fit. TalentFinder automates that first pass. You describe the role once — title, must-have skills, minimum experience, location — and the system pulls candidate profiles, reads each one, and returns a ranked shortlist where every candidate carries a match percentage, a breakdown showing *why* that number, and the specific skills they have and lack. The output exports straight to CSV.

4	2	358	~5,800
SCORING DIMENSIONS	LIVE DATA SOURCES	AUTOMATED TESTS, ALL PASSING	LINES OF BACKEND CODE

What live use has actually shown

These are not projections. They are the **63 live LinkedIn searches** run during development and testing:

OUTCOME	COUNT	SHARE
Completed and returned results	43	68%
Failed — account restricted by LinkedIn	4	6%
Failed — LinkedIn slow, or the connection dropped	4	6%
Failed — defects, since fixed	15	24%

A successful run took about **five minutes** for 15 profiles, at **20–48 seconds each**. Of 15 reviewed in one representative search, **3 were kept** — the filters do real work rather than passing everyone through.

HOW TO READ THE 68%

The defect share is genuinely closed — most notably a bug that discarded every profile already collected when a run failed part-way, which is now impossible. But the restrictions and the network failures are not ours to fix. They are the running cost of depending on a site that does not want to be read this way, and the number to plan against is the one measured here, not an improved one.

Matching accuracy, measured rather than claimed

The matching engine was re-run over the **237 real profiles already collected by the system** — genuine auditors, finance managers and employers, not fixtures. These are the results, and they are the strongest evidence in this document:

WHAT WAS MEASURED	RESULT	MEANING
Off-target candidates rejected search: external audit manager	**165 of 237** 70%	The old engine returned all 237. Every one of the twelve highest-ranked survivors is now a career external auditor.
Employer filter correctness Big Four selected	**33 of 33** 100%	Every candidate returned genuinely works at a Big Four firm, spread properly across all four. 39 wrong-firm employers were excluded, including respected practices like Mazars, RSM and Grant Thornton.
Score spread across the shortlist	100 / 81.2 / 43.8 max / median / min	23 score above 90, 20 below 70. The shape matters more than the average: a firm top band, and a tail the recruiter reaches last.

The rejection rate tracks how specific the request is — 79% for *financial reporting supervisor*, 70% for *external audit manager*, 62% for *finance manager*. It is not rejecting more of everything; it is rejecting what does not fit.

> **THE ONE NUMBER THIS DOCUMENT WILL NOT GIVE YOU**
>
> There is **no single accuracy percentage**, and any report quoting one would be inventing it. A percentage requires an answer key — a sample of candidates marked *would interview* or *would not* by a recruiter — and that exercise has not been done. It is about an hour of work and it is the first item in section 17. Until then, the specific measured results above are what can honestly be claimed.

The three things that matter most

1. The valuable part is the scoring engine, not the scraper. The system is deliberately built so the data source is a swappable component. The matching, ranking and explanation logic — the part that saves the recruiter time — works identically whether the profiles come from LinkedIn, from a paid data vendor, or from your own applicant database. Adding a new source is one class with two methods. That is the strategic point: the asset is source-agnostic, so it survives any decision the business makes about where to get data.

2. Every score is explainable and reproducible. There is no black box. A candidate scoring 90.0 has a stored breakdown showing exactly which component contributed what, and the scoring rules are versioned, so a score computed today can still be understood and compared six months from now. When a recruiter asks "why is this person ranked above that one?", there is a concrete answer. This matters commercially and it matters legally — automated decisions about people need to be defensible.

3. The LinkedIn scraping path is real, it works, and it is the riskiest component in the system. It breaches LinkedIn's terms of service and it gets the account it signs in with permanently restricted — **this has now happened twice, and the second one is current.** The code takes that seriously: a hard cap of 20 profiles per hour, manual human sign-in, no stored credentials, and an account-rotation path built for exactly this situation. But no amount of engineering makes it safe, and the honest recommendation is that it is a demonstration and a bridge, not the long-term supply. Section 15 covers this properly, and you should raise it yourself rather than let it be discovered.

WHERE THE PROJECT STANDS TODAY — LEAD WITH THIS, DO NOT BURY IT

The LinkedIn data path is down. The account it was using has been restricted by LinkedIn and cannot be recovered without submitting government ID. The system detected it, stopped the run, kept the profiles already collected, and marked the account unusable so nothing can reach it again.

A replacement account slot has been rotated in and is waiting to be signed into, which restores service. But that treats the symptom. The restriction happened at 20 profiles per hour *with* the full behavioural stack running — that is the ceiling telling us something, and **the next account is likely to go the same way.** Everything else — scoring, ranking, the web app, export — is unaffected and running, because the data source is a plug. That is the architecture working as designed, and it is the argument for section 17.

Current state

AREA	STATUS	DETAIL
Search & scoring engine	COMPLETE	Scoring (v4), filtering, experience maths, keyword normalisation — all tested
Matching accuracy	MEASURED	70% of off-target candidates rejected; employer filter correct on 33 of 33. Three defects found by the measurement and since fixed — see section 6
LinkedIn filter automation	BUILT, UNVERIFIED	Drives LinkedIn's own filter panel for employer and location. Not yet confirmed on a live run; the Big Four guarantee does not depend on it
Web application	COMPLETE	Search form, live progress, ranked results, candidate detail, saved list, CSV/JSON export
Background job system	COMPLETE	Searches run in a worker, not the web request; progress streams live to the browser
Demo data source	COMPLETE	Runs the whole product with no account, no network, no cost — this is what you demo
LinkedIn source	DOWN NOW	Code works, but the account is restricted. Needs a replacement signed in; will break again on any LinkedIn redesign
Account rotation	COMPLETE	Restriction is detected, the run stops, the account is retired, and a fresh one is issued with a new browser fingerprint
Indeed source	BUILT, UNPROVEN	Wired end to end and tested, but its extraction cannot be verified without a paid employer account — see section 8
Employer / qualification / career-stage filters	COMPLETE	Restrict to named firms (e.g. the Big Four), require a credential, bound the graduation year
Purchased data (Apify)	RULED OUT	Implemented and tested, then withdrawn by business decision. Code retained, unused.
AI semantic matching	BUILT, OFF BY DEFAULT	Works; costs money per candidate; disabled unless switched on
Authentication	BUILT, BYPASSED IN DEV	Real JWT verification is implemented; local development runs with it disabled
Production deployment	NOT DONE	Containers and compose file exist; nothing is deployed to a real environment
Data retention / deletion	NOT DONE	Required before processing real candidate data at scale — see section 15

2. The problem this solves, and for whom

The user is a recruiter with a role to fill

A recruiter sourcing for one open role typically opens somewhere between fifty and several hundred profiles. The great majority are rejected within seconds — wrong seniority, wrong stack, wrong city, not enough years. That triage is repetitive, it is attention-expensive, and it is done inconsistently: the hundredth profile of the day does not get the same scrutiny as the first.

Three things go wrong in that manual process, and each maps to something the system does:

THE MANUAL PROBLEM	WHAT THE SYSTEM DOES ABOUT IT
It is slow. Reading, judging and note-taking on each profile is minutes of human attention.	Extraction and scoring are automatic. The recruiter's attention goes only to a ranked shortlist, not the raw pile.
It is inconsistent. Two recruiters — or the same recruiter at 9am and 5pm — rank the same two candidates differently.	Scoring is deterministic: identical inputs always produce an identical score. The rules are the same for candidate one and candidate five hundred.
It is unaccountable. "I passed on them" rarely comes with a recorded reason.	Every candidate stores a score breakdown, matched and missing skills, and written reasons. Rejections are recorded with a cause.

What it is not

Being precise about the boundaries prevents the owner forming an expectation the system will not meet:

- **It does not contact anybody.** No messages, no connection requests, no outreach. It reads and ranks; the recruiter does the human part.
- **It does not decide who to hire.** It orders a list and shows its reasoning. A human makes every actual decision — which is also the right posture legally.
- **It is not a CRM or an applicant tracking system.** There is a "saved candidates" list with notes, but pipeline management, interview scheduling and offers are out of scope.
- **It does not find candidates who are not in the data source.** It is only as good as the pool it is pointed at.

Why the "source-agnostic" design is a business decision, not a technical one

This is worth landing clearly, because it is the answer to the most dangerous question in the room — *"what happens when LinkedIn shuts this down?"*

The system defines a candidate source as an interface with exactly two operations: *search for candidate cards*, and *fetch one full profile*. Everything after that — normalising skills, computing experience, scoring, filtering, ranking, storing, exporting — has no idea where the data came from. Candidates are identified in the database by `(source, profile_url)`, never by anything LinkedIn-specific.

The practical consequence: if LinkedIn becomes unusable tomorrow, what is lost is one class of roughly 900 lines. The scoring engine, the pipeline, the database, the API and the entire web application are untouched.

Pointing the product at a licensed data vendor, at an ATS like Greenhouse or Lever, or at an internal candidate database is a contained piece of work rather than a rewrite.

> **THE SENTENCE TO HAVE READY**
>
> "The data source is a plug. The product is what happens after the plug — and that part doesn't care where the profiles come from."

3. What a user actually does — the journey

Walking the owner through the screens in order is usually more persuasive than an architecture diagram. This is the whole flow.

Step 1 — Describe the role

The recruiter fills one form. The fields, and what each one actually controls:

FIELD	EFFECT ON THE RESULT
Data source	Demo data (instant, free, no account) or LinkedIn (real, rate-capped, needs manual sign-in).
Job title REQUIRED	Drives the search query and 30% of the score.
Keywords REQUIRED	30% of the score. Matched against the candidate's headline and job titles — so role words ("vendor management") work, tools buried in a profile do not.
Critical skills	Different in kind: not scored at all. Missing any one of these *discards* the candidate regardless of score. Checked against both their skills list and their titles.
Location	10% of the score. Optionally a hard filter via the strict-location switch.
Minimum years	30% of the score, and a hard cut: below the minimum, the candidate is discarded.
Company, industry	Refine the query sent to the data source. Not scored.
Maximum results	Caps how many profiles are opened and how many results are stored. Also the main cost and time control.
Minimum match score	Score threshold below which a candidate is discarded. Defaults to 40.

> **WORTH POINTING OUT IN THE DEMO**
>
> The distinction between **keywords** (scored proportionally — having two of three is worth 67%) and **critical skills** (a pass/fail gate) is the thing recruiters immediately understand. It is the difference between "nice to have" and "do not show me anyone without this".

Step 2 — The search runs in the background

Submitting the form does not block the browser. The API records the search, immediately returns, and hands the job to a background worker. The browser is redirected to a live progress page that streams updates over a persistent connection: how many candidates were found, how many are being processed, how many have been processed so far, how many have been kept — and, when scraping, how much of the hourly budget is left.

That last one is a deliberate touch: if a search is about to stall because the hourly scrape limit is nearly spent, the page says so *before* it stalls, instead of looking frozen.

Step 3 — Read the ranked results

Results arrive ordered by match score. Each card shows the candidate's name, headline, company, location, total years of experience, a match percentage rendered as a ring, and the matched and missing skills. Opening a candidate shows the full extracted profile — about, experience history, education, the complete skill list — alongside the score breakdown by component and the written reasons.

> **A DESIGN DECISION WORTH MENTIONING**
>
> **A search is never silently empty.** If the filters are strict enough that no candidate survives, the system returns every candidate it scored anyway, still ranked, rather than an empty page. An empty result screen reads as "the product is broken"; a ranked list of near-misses reads as "loosen your criteria" — and it tells the recruiter something true about the market.

Settings — where the data source is managed

One screen, and worth showing because it is where the risk becomes visible rather than theoretical. It reports whether a LinkedIn session is stored, whether the account has been restricted and why, and how many accounts have already been retired. It also carries the action to retire the current account and connect a different one. The count of retired accounts is deliberately prominent: it is the number that tells you whether this data source is working or quietly failing.

Step 4 — Save and export

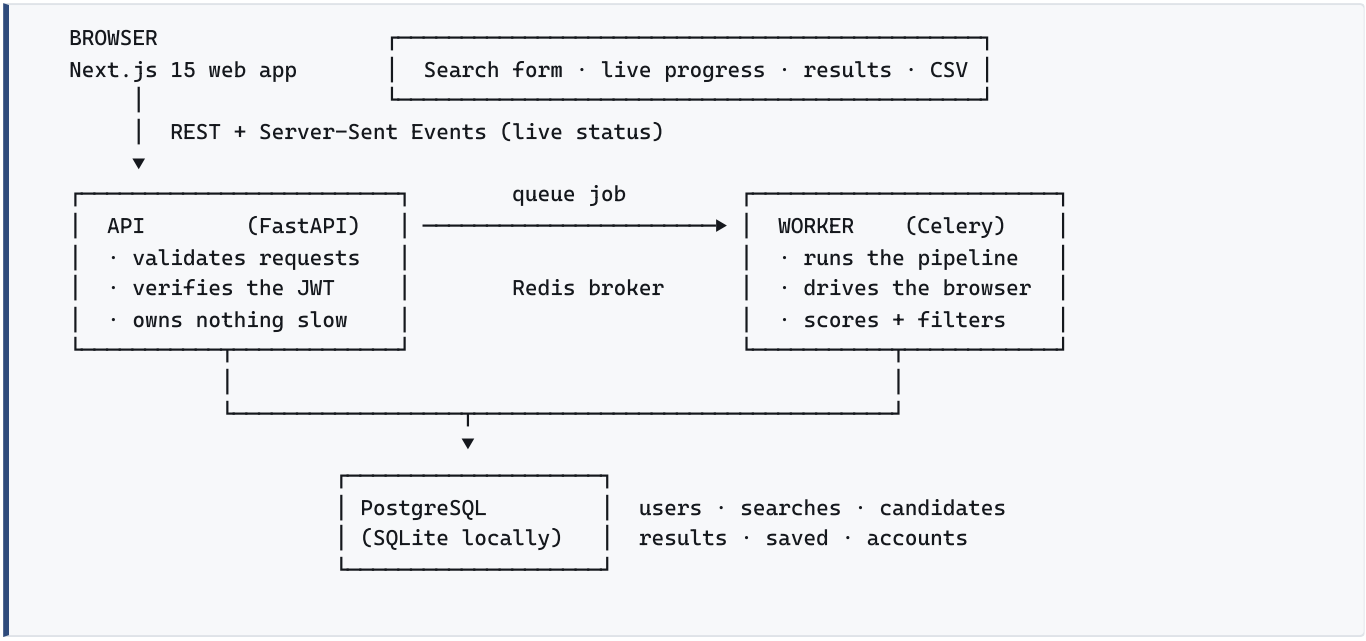
Candidates can be saved to a personal list with notes. Any search exports to CSV or JSON with one click — rank, name, headline, company, location, years, match score, matched skills, missing skills, and the profile URL. In practice the CSV is how the output leaves the system and enters whatever the business already uses.

The flow in one line

Describe	**Search**	**Watch**	**Review**	**Save**	**Export**
one form, once	runs in background	live progress	ranked, explained	shortlist + notes	CSV / JSON

4. Architecture at a glance

Four running pieces, plus a database and a queue. Each has one job.



Why the API and the worker are separate

This is the single most important structural decision, and it is worth being able to defend it in one breath: **a search takes minutes; an HTTP request should take milliseconds.**

Scraping a profile takes 30–60 seconds because it deliberately behaves like a human reading a page. Twenty profiles is therefore fifteen to twenty minutes. If that ran inside the web request, the browser would hang, any proxy would time it out, and one search would block the whole server. Instead the API writes a row, drops a job on a queue, and returns in milliseconds. The worker picks the job up and does the slow work. The browser follows along over a live stream.

It also isolates the risk. The scraping code, the browser binaries and the Playwright dependency live only in the worker image. The API never imports them. If the scraper crashes, the web application stays up.

The technology, and why each piece

LAYER	CHOICE	REASON IT WAS CHOSEN
Web app	Next.js 15 (App Router), React, Tailwind, shadcn/ui	Current mainstream stack; the component library gives a professional interface without a designer.
Server state	TanStack Query	Handles caching, refetching and loading states so the UI code stays about the UI.
API	FastAPI (Python)	Fast, typed, and generates its own interactive API documentation. Same language as the scraping and scoring code — one codebase, not two.
Validation	Pydantic v2	Every request and every provider response is validated against a schema at the boundary. Malformed data fails loudly at the edge instead of quietly corrupting the database.
Background jobs	Celery + Redis	The standard Python job queue. Survives restarts, retries, and scales by adding workers.
Database	PostgreSQL (SQLite for local dev)	Relational data with real constraints. SQLite locally means a developer needs nothing installed to run the app.
ORM & migrations	SQLAlchemy 2.0 (typed) + Alembic	Schema changes are versioned files, applied the same way in every environment.
Browser automation	Playwright + Chromium	Only used by the LinkedIn source. Isolated in the worker.
Authentication	Supabase JWT	Sign-in, password reset and session handling are someone else's problem; the API only verifies the resulting token.
AI	Anthropic Claude API	Optional semantic scoring layer. Off by default.
Packaging	Docker Compose, three images	One command brings up database, queue, API, worker and web app.

How the code is organised

The backend is layered so that the interesting logic has no dependencies:

FOLDER	CONTAINS	DEPENDS ON
`domain/`	Scoring, filtering, experience maths, skill matching, pre-filter	**Nothing.** No database, no network, no framework. Pure functions — which is why it is the most heavily tested part.
`providers/`	The three data sources plus the scraping behaviour layer	The domain models only. Providers extract; they never score.
`services/`	The search pipeline, candidate persistence, account handling	Domain + providers + database.
`api/`	HTTP routes	Services. Contains no business logic.
`db/`	Tables, enums, sessions	Nothing above it.
`workers/`	Celery app and task definitions	Services.

The rule that keeps this honest: **dependencies point inward only.** The scoring code cannot accidentally start making network calls, because it has nothing to make them with.

5. The search pipeline, step by step

This is what actually happens when a recruiter presses "Start search". If you can narrate these six stages, you can answer almost any question about system behaviour.

1. Search	2. Pre-filter	3. Fetch	4. Score	5. Filter	6. Store
get candidate cards	drop obvious misses	cache first, else open	four components	hard requirements	rank + persist

Stage 1 — Search: cheap cards first, filtered the way a recruiter filters

The source returns *cards*, not profiles: name, headline, company, location, URL. This is what a results list shows without opening anything. It is cheap and it is shallow — and that distinction drives the next stage.

Only the job title goes into the search box. Everything else — employer, location — is applied through LinkedIn's own filter panel, exactly as a recruiter would. This is not cosmetic. Text typed into the search box merely *biases the ranking*; a facet applied through the panel *filters server-side*, so pages of people who do not qualify are never returned and never cost budget.

> **THE SYSTEM PLANS ITS OWN FILTERS, AND KNOWS WHAT TO GIVE UP**
>
> Before searching, it decides which filters to apply and ranks them by how central each is to the request. If the filtered result set comes back too thin — fewer than half of what was asked for — it releases the *least* important constraint and looks again, reporting what it gave up rather than silently answering a different question.
>
> Employer is marked **never releasable**. A recruiter who asked for the Big Four does not want a longer list of people from elsewhere. Location is releasable, and is only applied as a hard filter at all when the strict switch is on — otherwise it stays a 10% scoring preference, because promoting it silently would hide candidates the recruiter said they would still consider.

Stage 2 — Pre-filter: discard only what is certainly wrong

Before opening any profile, cards are filtered. This exists to save money and, on the scraping path, to spend less of a scarce hourly budget on people who were never candidates.

The important design choice is that this filter is **deliberately timid**. It discards a card on only two signals, and only when both sides are present and clearly contradict: an explicitly required location the card contradicts (unless either side says remote), and an explicitly required company the card contradicts. Everything else is kept.

> **WHY TIMID IS CORRECT HERE**
>
> Card data is shallow. A strong candidate whose headline undersells them — "Engineer at Acme" for someone with ten years and the exact stack — must still be opened and scored properly. A false negative at this stage is invisible and unrecoverable: that person never appears, and nobody ever knows they were skipped. A false positive merely costs one page load. The asymmetry is not close, so the filter errs heavily toward keeping.

Stage 3 — Fetch: check the cache before spending anything

For each surviving card the system first looks in its own database for that person. If they were fetched within the freshness window — seven days by default — the stored copy is reused and **nothing is spent**: no page is opened, no hourly budget consumed, no vendor charge incurred, no additional exposure.

This compounds. A recruiter running three similar searches in a week pays the collection cost for overlapping candidates once. On the scraping path it is also a safety feature: fewer page loads is directly less risk.

Only on a cache miss does the source open the actual profile and extract it into structured fields: about, full experience history with dates, education, the complete skill list, certifications, languages.

Stage 4 — Score

The extracted profile is scored against the criteria on four weighted components, producing a 0–100 match score plus a stored breakdown. This is section 6, in full.

Stage 5 — Filter: the hard requirements

Scoring produces a ranking; filtering produces a yes or no. A candidate is discarded if they fail *any* of:

- Total experience below the stated minimum.
- Any **critical** skill missing.
- Match score below the recruiter's threshold.
- Location mismatch — but only when strict location is switched on.

Each failure records a human-readable reason, so a discard is never mysterious.

Stage 6 — Store

Survivors are sorted by score, assigned a rank, and written to the database along with the score breakdown, matched skills, missing skills and reasons — and the version of the scoring rules used. And here the never-empty rule applies: if nothing survived the filters, every scored candidate is stored instead, still ranked.

Two behaviours worth knowing by heart

> **PARTIAL RESULTS BEAT FAILED SEARCHES**
>
> If the hourly scrape budget runs out mid-search, the system does **not** throw the work away. It stops cleanly, keeps everything collected so far, records that it was rate-limited and when the next slot frees up, and completes the search with a partial ranked list. Because fetched profiles are cached, re-running the search later picks up where it left off rather than starting over.

> **ONE BAD PROFILE DOES NOT KILL A SEARCH**
>
> If a single profile fails to load — deleted account, a page that will not render, a changed layout — that one candidate is logged and skipped, and the search continues. Only an error that affects the whole run stops it. A search of twenty that hits one broken profile returns nineteen results, not an error page.

Progress is committed as it goes

Counts are written to the database at each step rather than at the end, which is what makes the live progress bar reflect reality. It also means a crashed worker leaves behind a record of exactly how far it got.

6. The matching engine — the core intellectual property

Everything else is plumbing. This is the part that produces the number the recruiter actually acts on, and it is where you should expect the most detailed questions.

The formula

COMPONENT	WEIGHT	HOW IT IS COMPUTED
Title	30%	Split into *what the job is* (the domain) and *how senior it is* (the level), scored separately. The domain is a gate: no evidence of the work means zero, however senior the candidate. Domain is judged across the whole career; seniority from the current role only.
Keywords	30%	Fraction of the recruiter's keywords found in the candidate's *headline and job titles*. Two of three = 0.667.
Experience	30%	Total years divided by the minimum required, capped at 1.0. Exceeding the requirement earns full marks, never bonus marks.
Location	10%	Binary: any shared word between required and candidate location scores 1.0, otherwise 0. "Remote" on both sides matches. No requirement = full marks.

The four components are multiplied by their weights, summed, and expressed as a percentage. The weights are a **named, versioned profile** — currently `v4` — and every stored result records which version produced it. That is what makes a score from six months ago still meaningful after the weights are tuned.

Any component the recruiter left blank is dropped and its weight redistributed across the rest. Nobody is marked down for failing a requirement that was never set — which is what makes location, keywords and critical skills genuinely optional rather than optional in name.

> **WHAT CHANGED IN V2, AND WHY**
>
> v1 scored a **skills list** at 30%, gave experience 20%, and spent 10% on education. Three changes: keyword matching replaced skills-list matching, experience rose to 30%, and education was dropped to zero.
>
> Education went because presence of a degree almost never decided an outcome — it was a tiebreaker consuming a tenth of every score, and that weight does more work behind demonstrated experience. Skills-list matching went for a sharper reason, covered below: on LinkedIn the Skills section is frequently incomplete, so scoring it punished people for not maintaining a list rather than for lacking the ability.

> **WHAT CHANGED IN V3 — THE MOST IMPORTANT FIX IN THE ENGINE**
>
> A search for **external audit manager** was returning **Finance Managers**. The cause was that both titles share the word "manager", and v2 scored that as a partial match on word overlap. It is not a partial match. It is a different profession.
>
> "Manager" describes a *level*. It appears in almost every title in every hierarchy and says nothing about what a person actually does. So v3 separates the two. **Domain** decides whether this is even the right job; **seniority** decides whether it is the right rung. A candidate matching only on seniority is not a near miss — they are rejected.
>
> Two further judgements come with it. Domain is read across the candidate's *entire career* — every job title held, plus employers — so an *Audit Manager* whose history is audit roles at an audit firm is recognised as an auditor whatever their current line reads. And **depth** separates a career auditor from someone who audited once a decade ago: both show the word, only one does the job.

Measured against 237 real profiles

The change was verified by re-running the engine over every profile the system has actually collected — not fixtures:

SEARCH	RETURNED BEFORE	RETURNED NOW	REJECTED
external audit manager	237	72	**70%**
financial reporting supervisor	237	49	**79%**
finance manager	237	91	**62%**

The most specific request cuts hardest and the broadest cuts least, which is the pattern that shows the gate is discriminating rather than simply rejecting. Full detail is in the companion document, *Matching Accuracy, Measured*.

TWO DEFECTS THE MEASUREMENT FOUND — AND WHAT V4 DID ABOUT THEM

Measuring found two faults in the ranking. Both are now fixed and re-measured. Tell this story rather than hiding it: it demonstrates the measurement is real work, not a presentation.

1. A self-declared skill could satisfy the job requirement. Six of 39 candidates cleared the domain gate on a LinkedIn *skill tag* reading "External Audit" with no supporting job history — among them a Finance Manager, a Head of Finance and a Group Head of *Internal* Audit. A skill tag costs nothing to add. The skills list is now excluded from the domain gate while remaining available to keyword matching, where self-declaration is fair evidence. **Result: 6 → 0.**

2. Internal auditors ranked too highly on an external audit search. A candidate with five internal audit roles and one external role from early in his career scored **100, joint first**, above people doing external audit at PwC today. Depth credited a role for matching *any* word of the request, so "audit" matched all five internal roles in full while "external" — the word separating the professions — counted for nothing. Each role is now credited by the share of the request it evidences. **Result: he moved to 79.2, thirteenth**, and the twelve above him are all career external auditors.

3. Audit managers at the Big Four were discarded entirely. The most serious of the three, and it only surfaced when each claim made about the engine was tested individually against the archive. A search for *external audit manager* was rejecting Audit Managers at KPMG, PwC, EY and Deloitte — 28 people at the exact firms the recruiter asked for. The cause is a fact about the profession, not a coding error: *at an audit firm, external audit is simply called "Audit"*. Only people distinguishing themselves from internal auditors write the word. An unqualified audit or assurance role now reads as external audit unless the career says "internal". **Result: Big Four candidates went from 16 to 33**, spread across all four firms instead of 11 from one PwC office.

All three fixes are in one file, covered by 15 new automated checks pinning the exact cases — including the internal-audit rejections, so the gate cannot be loosened later without a test failing. Released as `v4`.

A worked example — carry this one into the room

The role: Vendor Manager · Cairo · minimum 3 years · keywords *vendor management*, *procurement*, *contract negotiation*.
The candidate: headline "Senior Vendor Manager | Procurement & Supplier Strategy", current title "Senior Vendor Manager", a previous role titled "Procurement Specialist", 7.2 years of experience, based in Cairo, Egypt.

COMPONENT	WORKING	SUB-SCORE	CONTRIBUTION
Title	Required words {vendor, manager}; both present in the title → 2 of 2	100.0	0.30 × 1.000 = **0.300**
Keywords	"vendor management" matches *Vendor Manager* (manager ≈ management); "procurement" matches the past job title; "contract negotiation" appears in no title → 2 of 3	66.7	0.30 × 0.667 = **0.200**
Experience	7.2 years against a 3-year minimum, capped at 1.0	100.0	0.30 × 1.000 = **0.300**
Location	"Cairo" appears on both sides	100.0	0.10 × 1.000 = **0.100**
		Match score	**90.0%**

The recruiter sees **90.0%**, the breakdown above, "Missing: contract negotiation", and reasons reading *"Job title strongly matches" · "7.2 years experience (meets 3 required)" · "Title mentions vendor management, procurement" · "Location matches".*

The line to draw attention to is the keyword row: **the recruiter typed "vendor management" and the candidate's title says "Vendor Manager".** Those are different words, and the system treats them as the same thing.

Experience is calculated properly, and this is a genuine differentiator

Most naive implementations take the earliest start date and subtract it from today. That is wrong twice over: it counts career breaks as experience, and it double-counts overlapping roles — someone who contracted for two clients simultaneously for two years gets four years' credit.

This system parses each role's dates into an interval, **merges overlapping intervals**, and sums only the covered time.

```
Role A   2016-01 ──────────── 2019-06
Role B                2018-03 ──────────── 2021-09   (overlaps A)
Role C                                              2023-01 ── present (gap before)

Naive  (2016-01 → today)                        = 9.6 years   ← wrong
Actual (merged intervals, gap excluded)         = 8.2 years   ← what the system reports
```

The date parser handles the range of formats real profiles contain — "Jan 2020", "January 2020", "2020-03", "2020", "Present", "Current" — and simply ignores a role it cannot parse rather than guessing.

Keyword matching — where the accuracy actually comes from

Keywords are matched against **the candidate's headline, current title, and the title of every role they have held**. Not the About paragraph, not role descriptions, and not their skills list.

That scoping is a deliberate precision choice. A term buried in a paragraph is weak evidence — people write "exposure to procurement" about a single project. The same term in someone's *job title* is strong evidence, because a title is a claim their employer and their network can see.

Matching compares words rather than raw text, because raw text fails on exactly the cases that matter: **"vendor management" is not a substring of "Senior Vendor Manager".** Each keyword's words must all be

present, and words count as the same when they share a long enough opening — which is what lets ordinary word-form drift through:

RECRUITER TYPES	CANDIDATE'S TITLE SAYS	RESULT
vendor management	Senior **Vendor Manager**	✓ manager ≈ management
engineering	Software **Engineer**	✓
finance	**Financial** Reporting Lead	✓
data analysis	**Data Analyst**	✓
vendor	Head of **Vendors**	✓ plural folded
vendor management	Marketing Manager	✗ only one word present
data	**Data**base Administrator	✗ shared opening too short

Seniority words — senior, junior, lead, head, chief, principal, staff — are ignored on both sides, so a keyword is judged on the role rather than on the rung.

> **THE TRADE-OFF, STATED BEFORE ANYONE FINDS IT**
>
> This accepts some **over-matching**: "product" also satisfies a search for "production", because they share a long opening. That direction is chosen deliberately. In sourcing, a missed candidate is *invisible* — nobody ever learns they were skipped — while a loosely-matched one costs a recruiter about two seconds to skim past. When the two errors are that asymmetric, you tune toward recall.
>
> The narrower cost: a genuine skill that appears *only* in someone's Skills list, and never in a title, now scores zero. For title-driven roles — vendor manager, procurement, operations — that is correct. For a role defined by tooling rather than title, put the tool in **Critical skills**, which does check the skills list, or switch on the AI layer.

Design choices behind the numbers

- **Title and keywords carry 60% together.** Both read the candidate's stated role, from two angles: the title component asks "is this the job we are hiring for?", the keyword component asks "do the specifics we care about show up in their history?" Together they are what a recruiter actually screens on.
- **Experience carries 30%, up from 20%.** It is the hardest signal on a profile to overstate — dates are checkable and it is computed, not claimed.
- **Experience is capped.** Twenty years against a five-year requirement scores the same 100% as six years. Overqualification is not a better match, and without the cap it would distort every ranking.
- **Absent requirements score full marks, not zero.** If the recruiter states no location, every candidate gets the full 10% rather than everyone losing it. A criterion you did not ask for should not penalise anyone.
- **Missing keywords are surfaced, not hidden.** The gap is often the most useful output — it is what the recruiter probes in the first call.
- **Critical skills stay pass/fail and are checked more widely.** They are not scored at all; they discard. Because a hard filter should only reject when it is sure, they are checked against the skills list *and* the titles — rejecting someone titled "Vendor Manager" for not listing vendor management would be indefensible.
- **Scoring is a pure calculation.** It touches no database and makes no network call. That is why it can be tested exhaustively and why it is fast enough to be irrelevant to total runtime.

7. The AI layer

Optional, off by default, and deliberately narrow in scope. Understanding *why* it is narrow is the point.

What it does

When enabled, each scored candidate is additionally sent to Claude with the role requirements and a condensed profile. The model is instructed to judge *semantic equivalence* rather than keyword overlap — Backend Engineer ≈ Backend Developer, Node.js ≈ JavaScript backend, FastAPI ≈ Python backend — and to return a strict JSON object with a semantic score from 0 to 100, short reasons, and a list of qualities the candidate lacks.

The result is **blended evenly with the deterministic score**: the final number is half the rule-based score and half the model's. The model's reasons are merged into the displayed reasons, and the recorded score version becomes `v1+ai` so it is always visible after the fact which scores had AI involvement.

Why blend rather than replace

> **THE ENGINEERING ARGUMENT**
>
> A pure language-model score is not reproducible, cannot be explained component by component, and can drift when the model is updated. A pure keyword score misses obvious equivalences. Blending keeps a deterministic, auditable floor under every result while letting the model correct the specific failure it is good at correcting. Half the score is always something you can show your working for.

How it fails

Every AI call is wrapped so that **any** failure — API down, no credit, malformed response, timeout, non-JSON output — is caught, logged, and the candidate keeps its deterministic score. The search continues. The AI layer can never break a search; the worst case is that it silently does not contribute.

The cost, stated plainly

It is **one API call per candidate**. A twenty-candidate search is twenty calls. The configured default model is the most capable — and most expensive — option. For a task this well-defined, a mid-tier model is very likely to perform indistinguishably at a fraction of the price, and switching is a one-line configuration change.

> **RECOMMENDATION TO PUT ON THE TABLE**
>
> If AI matching is switched on for production use, change the model to a cheaper tier first and measure whether ranking quality actually differs. This is a single environment variable, and it is the difference between a rounding error and a line item.

What it deliberately does not do

- It does not write outreach messages or contact anyone.
- It does not make the accept/reject decision — the hard filters remain rule-based, so a model cannot admit a candidate who fails a stated requirement.
- It does not see the full raw profile: the about section is truncated and only relevant fields are sent, which limits both cost and unnecessary data exposure.

8. Where the data comes from — three interchangeable sources

All three implement the same two-method interface. Switching is a configuration change, and the rest of the system cannot tell the difference.

SOURCE	COST	ACCOUNT RISK	SPEED	BEST USED FOR
Demo DEFAULT	Free	None	Instant	Demonstrations, development, the entire automated test suite. Deterministic fixtures — no network, no account, no key.
LinkedIn HIGH RISK	Free in cash	**Severe** — permanent restriction of the account used	30–60s per profile, capped at 20/hour	Real data today, at small volume, accepting the risk. Breaches LinkedIn's terms.
Apify RULED OUT	Per profile, paid	None to your accounts	Minutes per batch	Real data without connecting a LinkedIn account. Built and tested; needs a token and is not currently offered in the UI.

Demo data

Deterministic fixture profiles loaded from disk. This is the default and it is what the entire test suite runs against, which is why the tests are fast and never flaky. For a demonstration to the owner this is almost certainly what you should use: it is instant, it cannot fail on stage, and it exercises every part of the product except collection.

LinkedIn — collected by us

A real Chromium browser is driven on the server. The operator signs in by hand; the application never types or stores a password. The session is saved encrypted and reused, so sign-in should happen once rather than per search. Extraction was rewritten against LinkedIn's current page structure. Section 9 covers the whole stack.

Indeed — collected by us, from the employer side

Added as a second live source, sharing all the machinery the LinkedIn provider uses: manual sign-in, no stored credentials, its own hourly cap, its own stored session, its own browser fingerprint. Adding it was a contained piece of work, which is the provider abstraction doing its job.

> **INDEED IS NOT A SECOND LINKEDIN — SAY THIS BEFORE ANYONE ASSUMES IT**
>
> On LinkedIn, any signed-in member can view member profiles, which is why reading them works at all. Indeed's candidate database is a **paid employer product**: it needs an employer subscription, candidates opt in to be searchable, and contacting them is metered separately. A job-seeker login cannot search resumes at all.
>
> Two consequences. First, **the integration cannot be verified without buying that subscription** — its extraction selectors are written against Indeed's documented structure but have never met a live employer session, so the first real search is a calibration run that writes the page to `_debug/`. Second, coverage is strongest in the US, UK and Canada; for finance and audit professionals in Riyadh, Jeddah and Cairo the population is likely a fraction of LinkedIn's. It should be treated as an unproven second source, not as the answer to the capacity ceiling.

Apify — bought from a vendor (ruled out)

Apify is a marketplace of hosted scrapers. The system calls their API, an "actor" runs on their infrastructure, and structured profile data comes back. **No LinkedIn account of yours is involved**, so there is nothing of yours to restrict.

Two details worth carrying:

- The configured actors are **cookieless** by design. Some available actors accept a LinkedIn session cookie to reach more data — using one of those would reintroduce exactly the account-ban risk this path exists to avoid. That constraint is documented at the configuration site so nobody swaps it in casually.
- **Treat the vendor as rented, not owned.** Scraping vendors get sued by LinkedIn and disappear — Proxycurl, a well-known one, was sued and shut down in 2025. The provider abstraction is what makes that survivable rather than fatal.

> **THE STRATEGIC FRAMING, WITH ONE OPTION NOW REMOVED**
>
> There are three ways to get this data: **collect it yourself** (free, highest risk, does not scale), **buy it** (predictable cost, no account risk, vendor dependency), or **license it officially** — LinkedIn's own recruiting product, or integrating with an ATS.
>
> **Buying it has been ruled out by business decision.** That was the inexpensive middle path and it was already built and tested, so removing it narrows the choice sharply: either accept that self-collection caps the product at roughly one recruiter, or fund a licensed source. There is no longer a cheap route to outbound sourcing at volume, and that should be understood as a deliberate trade rather than discovered later.
>
> The architecture remains ready for whichever is chosen — that readiness is the deliberate output of the design, and it is why this is a procurement conversation rather than a rebuild.

9. The scraping stack, and why it is built the way it is

This section only applies to the LinkedIn source. It is the most technically involved part of the system, and the part where it is easiest to sound either naive or reckless. The framing that is both accurate and defensible: **these are four layers in descending order of how much they actually protect you, and the first one matters more than the other three combined.**

Layer 1 — The volume cap. This is the control that matters.

At most **20 profiles per rolling hour**. Not 20 per search, not 20 per day — 20 in any sixty-minute window, counted across every search and every restart.

The reasoning is simple and it is the thing to lead with: **volume is what abuse detection actually keys on.** An account opening hundreds of profiles an hour is caught no matter how convincing its mouse movements are. Realistic behaviour is a distant second to not doing very much.

> **THE DETAIL THAT SHOWS THE THINKING**
>
> The window is **persisted to disk**, not held in memory. An in-memory counter resets whenever the worker restarts — so a crash loop, or an impatient operator re-running a search, would quietly hand out a fresh budget of 20 every time and defeat the entire limit. Writing timestamps to a file means the budget survives restarts. The file is written atomically, because a half-written file would read back as an empty budget and produce exactly the failure it was meant to prevent.
>
> If a search exhausts the budget it stops cleanly and keeps what it collected, rather than failing or waiting indefinitely.

Ceiling implied by this: 20 profiles/hour is roughly 160 per eight-hour working day per connected account. That is the honest capacity number, and section 14 builds on it.

Layer 2 — Manual sign-in

The application never types a password and stores none. A visible browser window opens and waits — up to ten minutes — for a human to sign in and clear any CAPTCHA. The resulting session is encrypted and reused, so this should happen once, not per search.

Three benefits, and the third is the one people miss:

- Login and CAPTCHA are the most heavily scrutinised moments, and they are performed by an actual human, because they are.
- A stolen copy of the database yields no LinkedIn passwords, because there are none.
- It forces an honest architecture: the system genuinely cannot run unattended at volume, which is a feature given what unattended volume would do to the account.

If LinkedIn restricts the account, the code fails **fast and terminally** with an explicit message rather than retrying. A restriction requires government-ID verification to clear; retrying is pointless and makes things worse.

Layer 3 — Human behaviour emulation

Automation is normally obvious because it is *too perfect*: instantaneous cursor jumps, pixel-exact clicks, identical delays, flawless typing. This layer removes those signatures. It is split into pure mathematics — testable with no browser involved — and a thin layer that drives a real page with it.

BEHAVIOUR	TECHNIQUE
Mouse	Curved Bezier paths rather than straight lines, sampled at a constant time step so velocity varies naturally. Movement duration follows Fitts's law — the real human relationship between distance, target size and time. Every movement overshoots the target by 8–12% and then corrects, which is what a hand actually does.
Scrolling	Bursts whose deltas decay geometrically while the gaps between them grow — the signature of a flick with inertia — followed by small adjustments once the target is near, sometimes correcting backwards.
Timing	Log-normal delays, not uniform random: a floor, a common case and a long tail, which is the shape human reaction times actually have. Micro-breaks every six or so actions. A hesitation between the pointer arriving and the click landing.
Typing	A per-session words-per-minute rate with Gaussian variation, longer pauses at word and sentence boundaries, and a 1.5% typo rate — where the typo is noticed after a beat, backspaced, and corrected.
Safety	A ten-point honeypot check before any click: display, visibility, opacity, size, off-document position, negative stacking, aria-hidden, suspicious naming, hidden or untabbable inputs, clipped or inert elements. Honeypots are invisible traps that only automation clicks — clicking one is a confession.

Layer 4 — Fingerprint consistency

The insight here is worth stating precisely, because it is the opposite of what most people assume. Anti-bot systems rarely catch a single unusual value. **They catch disagreement between values that a real browser keeps in sync.** So the goal is not to look exotic or to hide — it is to be internally consistent.

- **Automation flags removed.** The `webdriver` marker is set back to false rather than deleted — real Chrome *has* that property, so removing it entirely is as distinctive as leaving it true.
- **Patches target the prototype, not the object.** Defining properties directly on the browser's navigator object leaves own-properties where a real browser has none, and listing those properties is a cheaper bot check than reading any single value.
- **The claimed Chrome version is read from the live engine**, never hard-coded. Claiming a newer Chrome than the binary actually is fails ordinary feature detection instantly.
- **Per-account machine identity.** GPU, CPU, memory, display and OS build are derived from the stored account's database id and drawn as coherent sets — no four-core laptop claiming a high-end graphics card. Same account, same machine forever; a different account gets different hardware. This matters because device correlation is how platforms link accounts: a single fixed fingerprint means a restriction on one account follows you to the next.
- **WebRTC leak prevention** at both the launch and the connection level — while leaving the API present and functional, because its absence is itself a signal.

> **STATE THIS BEFORE YOU ARE ASKED**
>
> **The IP address is not mitigated.** Every request comes from the host machine and there is no proxy support. After a restriction, the IP is the single strongest remaining link between one account and the next. The fingerprint work defeats *identical-device* matching; it does not defeat a determined correlation effort, and it does not make traffic anonymous.

When it happens: the restriction and rotation lifecycle

Assume a restriction, because one has occurred. What the system does about it is worth describing, because the failure it prevents is subtle and expensive.

STAGE	BEHAVIOUR
Detect	The lock is recognised from the page the browser lands on and reported as a restriction — never as "a CAPTCHA" or a selector bug, which would send someone debugging a problem that does not exist.
Stop	The whole run halts at once. It is treated as terminal in its own right, not as the failure of one profile.
Keep	Profiles already collected are stored and ranked *before* the failure is raised. A restriction costs the rest of the run, never the work already paid for.
Retire	The account is marked restricted and its stored session deleted. Later searches are refused *before a browser opens*, so nothing can reach the locked account again.
Rotate	From Settings, the operator retires the account and is issued a fresh one. The replacement gets a new identity rather than a reset of the old one.

WHY ROTATION ISSUES A NEW RECORD INSTEAD OF CLEARING THE OLD ONE

The browser fingerprint is derived from the stored account's row id (layer 4 above), so clearing the session and reusing the same record would hand the replacement account the *burned account's machine identity* — the strongest correlation signal after IP, and exactly how platforms link one account to the next. Rotation therefore creates a new record, and retired records are never deleted, so an identity is never handed out twice.

THE LIMIT OF ROTATION — SAY THIS IN THE SAME BREATH

Rotation defeats *device* correlation. It does nothing about the IP address, and a fresh account signing in from the address that just had one restricted is the strongest remaining link. Settings shows a running count of retired accounts precisely so the pattern stays visible: **if that number climbs, rotation is not working, and the answer is a different data source rather than a third account.**

Extraction: why it broke before, and what changed

An earlier version of this scraper failed in a specific and instructive way. It located page elements by their CSS class names — but LinkedIn generates those names per build, so they rotate on every deploy. When they rotated, extraction silently returned names with nothing else. Every candidate then scored the same flat low number, and the failure looked like a scoring bug rather than a collection bug.

The rewrite anchors on things that carry meaning instead:

- **Component keys** — LinkedIn's own server-driven UI contract, which identifies a section by what it *is* rather than how it is styled.
- **Entity collection containers** for individual entries.
- **Test identifiers and visible heading text** as fallbacks.

Two structural problems were solved along the way. First, experience entries come in two shapes — a grouped employer containing several roles, and a standalone entry — and the line above the date range means *job title* in one and *company* in the other. Flat text parsing cannot tell them apart; only the container structure

disambiguates. Second, the sections below the activity feed load lazily in batches as you scroll, so the reader scrolls until they appear rather than a fixed number of screenfuls.

> **SAY THIS PLAINLY**
>
> **When LinkedIn redesigns, this will break again.** That is not a defect in the implementation; it is the nature of depending on someone else's private page structure. What the code does about it: on a failed extraction it writes the complete page and a screenshot to a debug folder. That dump is how the current version was derived, and it turns re-tuning into an afternoon rather than a rediscovery. This is a permanent maintenance cost that comes with the scraping path, and it should be priced in when the owner asks what ongoing effort looks like.

10. Data model and storage

Six tables. The structure carries a few decisions worth being able to explain.

TABLE	HOLDS
`users`	Identity, mirroring the authentication provider's user id.
`provider_accounts`	A user's linked data-source accounts — one live, plus every one retired. Session state is stored **encrypted**; the row id seeds the browser fingerprint, which is why retired rows are kept rather than deleted.
`searches`	The criteria, the source used, the scoring version, status, live progress counters, and any error.
`candidates`	The extracted profile: name, headline, title, company, location, about, experience, education, skills, certifications, languages, computed total years, and when it was fetched.
`search_results`	The join between a search and a candidate: match score, score version, breakdown, matched and missing skills, reasons, rank.
`saved_candidates`	A recruiter's personal shortlist with notes.

Decisions embedded in the schema

- **A candidate is identified by (source, profile URL)** — never by anything LinkedIn-specific. This is the same source-agnostic principle expressed in the database. A uniqueness constraint on that pair means the same person collected twice is one row, updated, rather than a duplicate.
- **Candidates are shared; results are per-search.** One stored profile serves every search that finds that person. This is what makes the seven-day cache work, and it is why a second similar search is dramatically cheaper than the first.
- **Scores are stored, not recomputed.** Each result keeps its score, its breakdown and the rules version that produced it. Re-opening a search from March shows what it actually showed in March, even after the weights change.
- **Total years is computed once at extraction** and stored, rather than recalculated on every read.
- **Deleting a user cascades** to their searches, results and linked accounts, which is the foundation of a workable deletion path.

> **A PRODUCTION GAP TO NAME YOURSELF**
>
> Each candidate row keeps a `raw` field holding the unprocessed source response, which is invaluable for debugging extraction. In production that is more personal data than the product needs. Before real use at scale this should be pruned or dropped, and a retention job should delete candidate data past its useful life. Neither exists yet — it is on the list in section 17, and it is small work.

11. Security, authentication and secrets

Authentication

Sign-in is handled by Supabase, which issues a signed token. The API verifies that token on every request and resolves it to a user. Both current Supabase project styles are supported — shared-secret verification and public-key verification against a published key set — so the deployment is not locked to one configuration.

For local development there is an explicit bypass that injects a fixed development user, so the whole application can be run end-to-end without configuring an auth provider. Two guards matter here: the bypass requires the environment to be marked as development, and it is refused when the environment is marked production. That combination is what stops a convenience feature becoming an open door.

Authorisation

Every search endpoint resolves the record *through* the current user, so requesting another user's search returns "not found" rather than their data. This is enforced at the point of lookup rather than as a separate check that could be forgotten on a new route.

Secrets and encryption

- Browser session state is encrypted at rest with a symmetric key held only in configuration, never in the database or the code.
- No LinkedIn credentials exist anywhere in the system — sign-in is manual by design, so there is nothing to steal. This is now structural rather than a convention: the endpoint that once accepted a password, and the database column that stored it, have both been removed, so there is no longer anywhere to put one.
- Logging is configured to avoid credentials, session state and full profile personal data.
- The local database, browser sessions and any raw debug dumps are excluded from version control.
- Cross-origin access is restricted to a configured list of allowed origins.

Input handling

Every incoming request is validated against a schema before it reaches any logic — numeric ranges are bounded, required fields enforced, unknown fields rejected. Database access goes through an ORM with parameterised queries, so SQL injection is not an available attack. Provider responses are validated on the way in as well, which is what stops a malformed profile from a third party corrupting stored data.

> **BEFORE PRODUCTION, THREE THINGS**
>
> (1) Set the environment to production and confirm the auth bypass is off — the code refuses it, but verify rather than assume. (2) Generate a fresh encryption key; the example configuration ships an obvious placeholder. (3) Serve everything over HTTPS with the allowed-origins list narrowed to the real domain.

12. Quality: what "358 tests" actually covers

"358 tests" means nothing on its own. What matters is *what* they cover, and the distribution here is deliberate: the heaviest testing is on the pure logic, because that is the part where a silent error produces a wrong answer that looks plausible.

AREA	TESTS	WHAT IS VERIFIED
Human motion & timing models	32	Bezier paths, Fitts's-law durations, log-normal delays, scroll inertia, typing plans with typo correction
Browser fingerprint derivation	26	Coherent hardware sets, stability per account, distinctness across accounts, version handling
Apify provider	28	Retained though the path is withdrawn; the tests still pass and document the mapping
Indeed provider	18	Factory wiring, URL and paging construction, page-state detection, date parsing, and that each platform holds a separate hourly budget
Employer matching	17	EY ↔ Ernst & Young, PwC ↔ PricewaterhouseCoopers, legal suffixes, and that a card with no employer is never discarded
Company facet ids	15	Reading LinkedIn's `currentCompany` ids from a real search URL, and word-bounded label matching so "EY" cannot tick "Honeywell"
Graduation year	22	Date formats, that the earliest graduation is what counts, and that a profile without dates is kept
LinkedIn extraction	16	Both experience shapes, education, skills, date splitting — against synthetic fixtures reproducing the live structure
Rate limiter	11	Window enforcement, persistence across restarts, atomic writes, waiting and refusal behaviour
API endpoints	8	Create, read, results, export, ownership isolation
Experience calculation	14	Date formats, overlap merging, gap exclusion, unparseable input
Keyword matching	20	Word-form drift (manager/management, engineer/engineering), plurals, multi-word terms, seniority words, and that About text alone never satisfies a keyword
Scoring	11	Each component independently, the weighted total, the capped experience rule, and that an unstated requirement never penalises
Skill matching (critical-skills filter)	12	Normalisation, aliases, matched/missing sets, ratios
Pre-filter	6	That it discards only clear mismatches and keeps borderline cases
Filtering	5	Each hard requirement and the reasons recorded
Human actor on a page	5	Honeypot detection, click and scroll behaviour
Provider interface	5	Contract conformance, factory selection
Pipeline under rate limit	4	That an exhausted budget stops cleanly and keeps partial results

Provider accounts & rotation	7	Encrypted session storage, restriction marking, that every rotation yields a distinct fingerprint seed, retired-account history
Pipeline under restriction	5	That a restriction stops the run instead of skipping each profile, keeps partial results, retires the account, and refuses to start again until rotated
Encryption, configuration	6	Round-trips, environment parsing, defaults

VERIFIED WHILE WRITING THIS DOCUMENT

The full suite was executed while preparing this document: **358 passed** in 36 seconds. Every test runs against fixtures — no network, no LinkedIn, no browser. That is why they are fast and why they never fail for reasons unrelated to the code.

What is not tested, stated honestly

- **Live LinkedIn extraction.** Extraction is tested against synthetic fixtures that reproduce the real page structure, but nothing verifies that today's actual LinkedIn still matches. Nothing can, without scraping in a test — which would be both unreliable and exactly the thing the rate limit exists to prevent.
- **The frontend.** Type-checking runs, but there are no component or end-to-end browser tests.
- **Load and concurrency.** No performance testing has been done. The rate limiter's own documentation notes that multiple worker processes sharing one budget file can each land a slot in the same instant — the current design runs one, and if that changes the limit should be lowered rather than relied on to arbitrate.

No scraped profile is committed to the repository — the extraction fixtures are entirely synthetic. Storing real people's profile data in source control would contradict the project's own compliance documentation, and that consistency is deliberate.

13. Running it: deployment and operations

Local development

Neither a database server nor a queue is required. The backend falls back to a file-based database and runs jobs inline in the same process, so a developer clones the repository, installs dependencies and has a working application. Demo data means no account and no keys are needed for the full experience.

Containerised

A single compose file brings up five services: PostgreSQL, Redis, the API, the worker and the web application. The API and worker are separate images — the worker carries the browser and Playwright, the API does not — which keeps the API image small and means a scraping dependency cannot destabilise the web tier.

Operational things that will bite, and are already documented

SYMPTOM	CAUSE
Every candidate scores 0 on keywords	Keywords are matched against titles only. Terms that live in someone's skills list or About text will not match — use role words, or move the term to Critical skills (section 6)
Profiles stored with no headline or experience	Page structure drift. The debug dump written on failure is the starting point
The browser hangs on submit	Development mode runs the search inside the request. Expected; use a real worker for background execution
"LinkedIn has restricted this account"	Terminal. Requires government-ID verification with LinkedIn; no retry helps. Searches will refuse to start until a different account is connected from Settings
A LinkedIn search is refused before the browser opens	Working as intended — the connected account is marked restricted. Rotate to a new one in Settings
After rotating, nothing prompts for a login	Sign-in happens on the next LinkedIn search: start one with max results set to 1 and the browser window will open
The rate limit appears not to apply	The process was started from the wrong directory, so the budget file resolves elsewhere

THE SHARPEST OPERATIONAL EDGE IN THE SYSTEM

Four things resolve relative to the working directory: the configuration file, the database, the debug folder, and — critically — **the rate-limit budget file**. Start the worker from the wrong directory and it silently gets a fresh 20-profile budget, bypassing the one control that actually protects the account. It fails quietly rather than loudly, which is the worst way for a safety control to fail. This is documented in three places, but in production it should be an absolute path rather than a documented convention. Small fix; worth doing before any real use.

SCHEMA CHANGES

The database is versioned, so any environment picking up the account-rotation work needs `alembic upgrade head` run against it once. The migration adds account history and drops the unused credentials column; it has been tested in both directions.

What production still needs

- A real deployment target — nothing is hosted anywhere today.
- Managed PostgreSQL and Redis rather than containers.
- Log aggregation and error alerting. Logs are structured and ready for it; nothing collects them.
- A backup and restore policy for the database.
- Automated deployment on merge.
- A retention and deletion job for candidate data (section 15).

14. Cost model — what this costs to run

Have these numbers ready. "I'll find out" is a weak answer to the first question an owner asks.

Infrastructure

COMPONENT	ORDER OF MAGNITUDE	NOTE
Application hosting (API + worker + web)	Low tens of $ / month	Modest requirements; the worker needs enough memory to run a browser
PostgreSQL (managed)	Low tens of $ / month	Text and JSON only; data volume stays small
Redis (managed)	Single-digit $ / month	Queue only, not a data store
Authentication	Free at this scale	Supabase free tier covers a small team comfortably

Infrastructure is not where the money is. Data is.

Data collection — the real variable

SOURCE	COST PER PROFILE	CEILING
Demo	Zero	None — but it is not real data
LinkedIn	Zero cash · non-zero risk	**20/hour ≈ 160 per working day per account.** This is a hard product ceiling, not a tuning parameter
Apify	Cents per profile, per the vendor's pricing	Effectively unlimited; it is a budget question, not a technical one

> **THE CAPACITY CONVERSATION, IN NUMBERS**
>
> One recruiter sourcing one role typically wants 50–150 profiles reviewed. On the scraping path that is **three to eight hours of wall-clock time** against the hourly cap — and the cap is per *account*, not per user, so five recruiters sharing one connected account share one 20-per-hour budget between them.
>
> The seven-day cache genuinely softens this: overlapping searches reuse stored profiles at zero cost and zero risk. But the structural point stands and should be said out loud — **the scraping path does not scale past roughly one active recruiter.** Scaling means buying data or licensing it. That is a commercial decision, not an engineering one, and it is the most useful thing this document can put in front of the owner.

AI matching

One model call per candidate, only when enabled. A 20-candidate search is 20 calls with short inputs and short outputs. On the currently configured top-tier model this is a small but real per-search cost that grows linearly with usage; on a mid-tier model it is close to negligible. The recommendation from section 7 stands: switch the model tier before switching the feature on, and measure whether the ranking actually differs.

Engineering time — the honest line item

The recurring cost that is easy to omit from a budget: **the LinkedIn extraction will break whenever LinkedIn changes its page structure**, on their schedule, without warning. Each break is roughly an afternoon of work given the debug dumps — but it is unpredictable, and while it is broken the real-data path

produces empty profiles. A paid or licensed source moves that maintenance burden onto a vendor, which is part of what the money buys.

15. Legal and compliance — the honest section

> **RAISE THIS YOURSELF. DO NOT LET IT BE DISCOVERED.**
>
> Credibility in this conversation comes from being the person who already understood the risk and built controls around it — not the person who had it pointed out to them. The project ships a dedicated compliance document precisely so this is on the record rather than glossed over, and the code's default configuration is the safe one.

Scraping LinkedIn breaches its User Agreement

Section 8.2 of the LinkedIn User Agreement prohibits scrapers, bots and other automated access. Using the LinkedIn source breaches it. That is not a grey area and it should not be presented as one.

The three concrete consequences

- **Account restriction or permanent ban** of the account used. This is the likely one, it is terminal, and clearing it requires sending LinkedIn government identification. **It has now happened twice to accounts used with this project, and the second restriction is current** — this is not a theoretical risk we are describing, it is the observed outcome of running the tool.
- **Legal exposure.** Scraping has been litigated — *hiQ Labs v. LinkedIn* being the well-known case — and outcomes are fact-specific and have shifted over time. Accessing data *behind a login*, in breach of terms you accepted, is materially riskier than reading fully public pages.
- **Detection.** LinkedIn actively fingerprints automation. Nothing in section 9 makes this safe; it reduces the rate of detection. That is the accurate claim and the only one worth making.

Personal data obligations

Candidate profiles are personal data about identifiable people who have not been asked. Depending on jurisdiction — GDPR in Europe, CCPA in California, and others — processing it may require a documented lawful basis, a retention policy, a deletion and subject-access path, and in some readings notification to the individuals concerned.

What exists today: a freshness window that controls cache reuse, cascading deletion when a user is removed, no profile data in version control, and logging configured to avoid personal data. What does not exist: a purge job, a subject-access process, and a documented lawful basis. Those are organisational commitments as much as code, and they need a decision before real candidate data is processed at scale.

> **ON RECOVERING A RESTRICTED ACCOUNT**
>
> LinkedIn's verification checkpoint is the genuine route back, and completing it normally restores the account. Two things follow from that, and both matter commercially. It ties a named individual's government ID to the account permanently; and an account recovered that way should never be pointed at this tool again, because a second restriction on an identity-verified account is far harder to walk away from. **Recovering an account and continuing to scrape with it are mutually exclusive choices.**

The three paths, with their trade-offs

PATH	LEGAL POSTURE	COST	REALITY
Keep scraping	BREACHES TOS	Free in cash	20/hour, and burns an account roughly that fast — two lost so far. Breaks on redesigns. Does not scale. Every account used must be one we are prepared to lose.
Buy from a vendor	RULED OUT	Per profile	Was implemented and carried no account risk, but has been withdrawn by decision. Vendors do get litigated and disappear, so the concern is reasonable — the cost is that no inexpensive path to volume remains.
License officially	CLEAN	Highest	LinkedIn Talent Solutions, or ATS integrations for candidates already in a pipeline. Requires approval and a commercial relationship. The only durable answer.

What the code does to keep the safe option the easy one

- The default source is demo data — the entire product runs with no account, no network and no risk.
- Scraping is opt-in, per search, and isolated in its own worker so it can be removed without touching anything else.
- The volume cap is low by default and survives restarts.
- The compliance document is committed alongside the code and referenced from the configuration, so nobody enables scraping without passing the warning.

> **THE POSITION TO TAKE IN THE ROOM**
>
> "The scraping path is a working demonstration and a bridge for small volumes. It is not what we should build a business on, and the architecture was designed so we don't have to. Switching to a licensed or purchased source is a configuration change plus a commercial conversation — the product doesn't change. I'd like a decision on which source we're standardising on, because that decision is the gate on everything else."

This document is engineering analysis, not legal advice. A lawful basis and jurisdiction-specific obligations should be confirmed with counsel before processing real candidate data.

16. Known limitations and technical debt

Knowing your own weak points is more convincing than claiming there are none. These are ordered by how likely they are to come up.

CLOSED SINCE THE LAST REVIEW

A code-quality pass removed a dead endpoint that accepted and stored a LinkedIn *password* nothing ever read — along with its database column, which contradicted this project's own claim to store no credentials. The same pass fixed a bug where a detected account restriction was handled as "skip this profile", so a locked account was re-opened once per remaining profile in the search — the exact behaviour that turns a restriction permanent. The project linter now passes clean, having previously reported 44 violations.

Scoring moved to **v4** (title 30 / keywords 30 / experience 30 / location 10). v2 replaced skills-list matching — which scored zero for "Financial Accounting" against "Financial Reporting" — with keyword matching that tolerates word forms. v3 then split job title into domain and seniority, which is what stopped a Finance Manager scoring against a search for an external audit manager. v4 then closed two faults the accuracy measurement exposed: a self-declared skill tag could satisfy the job requirement, an internal auditor with one old external role ranked joint-first, and — most seriously — audit managers at all four of the Big Four were being discarded because their titles say "Audit" rather than "External Audit". Earlier results keep their version tag, so nothing already stored silently changes meaning.

The **Big Four guarantee** was closed. LinkedIn's employer filter runs server-side, so a missed click meant it silently did not apply — and cards showing no employer were deliberately kept, with nothing re-checking them afterwards. An External Audit Manager at a non-Big-Four firm could reach the shortlist. The employer is now verified a second time once the profile is open, where it is actually known, and an employer that cannot be read is rejected rather than assumed.

LIMITATION	SEVERITY	DETAIL AND MITIGATION
Keywords read titles only	MEDIUM	A term that appears solely in a candidate's skills list or About text scores zero, and keywords are 30% of the score. Deliberate — titles are the strong signal — but it means keyword wording matters. Put must-have tooling in Critical skills instead. *v1's harsher "literal skill matching" problem is resolved: word forms like manager/management now match.*
Scraping does not scale	HIGH	20 profiles/hour per account is a hard ceiling — roughly one active recruiter. Structural, not a tuning problem. Fixed only by changing data source.
Extraction will break	HIGH	Depends on LinkedIn's private page structure. Debug dumps make repair quick, but it is unplanned recurring work on someone else's schedule.
Binary location scoring	MEDIUM	Any shared word scores full marks; none scores zero. No concept of distance, region, or commuting range — "Berlin" and "Munich" are equally wrong.
No labelled accuracy benchmark	MEDIUM	Nobody has marked a sample of candidates *would interview / would not*, so there is no single defensible accuracy percentage — only the specific measured results in section 6. About an hour of recruiter time fixes this, and it is the first item in section 17.
Filter automation unverified live	MEDIUM	The code driving LinkedIn's filter panel has not completed a confirmed live run. Correctness does not depend on it — the employer check runs again after each profile is opened — but the budget saving does. Needs one verified search.
Accounts burn faster than they rotate	HIGH	Rotation restores service but defeats only *device* correlation. The IP is unchanged and there is no proxy support, so each replacement is more exposed than the last. Two accounts lost so far. This is a supply problem, not a bug to fix.
Raw profile data retained	MEDIUM	Useful for debugging, more personal data than the product needs. Should be pruned before production. *Still outstanding.*
Budget file is path-relative	MEDIUM	Starting from the wrong directory silently grants a fresh budget. Documented, but a quiet failure in a safety control. Should be an absolute path. *Still outstanding.*
No sign-in without a search	MEDIUM	After rotating, connecting the replacement account means starting a small search so the browser window opens. Settings has no dedicated sign-in action yet.
Live status stream expires early	MEDIUM	The progress connection gives up after 10 minutes, but a full 20-profile scrape takes longer, so the page stops updating before the search ends. Behaviour left as-is and documented rather than changed mid-review.
No frontend tests	MEDIUM	Type-checking only. A UI regression would not be caught automatically.
Indeed is unverified	MEDIUM	Built, wired and tested, but its extraction has never met a live employer session — that needs a paid subscription. Treat the first search as calibration.
Apify path withdrawn	MEDIUM	Implemented and tested with 28 tests, and now withdrawn by business decision rather than by any technical problem. The code and tests are retained; reversing the decision is a configuration change, not development.

Rate limit across processes	MEDIUM	Two workers sharing one budget file can each take a slot in the same instant. Fine at one worker; if scaling out, lower the limit rather than relying on the file to arbitrate.
Education scoring is crude	LOW	Presence-based only — no field of study, institution quality or relevance. At 10% weight this rarely changes an outcome.
Progress polls the database	LOW	The live stream re-reads the row roughly once a second per open page. Simple and adequate now; would need revisiting with many concurrent viewers.
Small config inconsistency	LOW	The per-search profile ceiling defaults to 25 in code and 20 in the example configuration. Harmless, but they should agree. *Still outstanding.*

17. Where I would take it next

Ordered by value per unit of effort. Having a considered order ready is what turns "it's built" into "here's the plan".

Immediate — days, and mostly independent of any strategic decision

1. **Fix the two quiet failure modes.** Make the rate-limit budget path absolute so it cannot be bypassed by a working directory, and reconcile the two differing profile-ceiling defaults. Both are small; the first is a safety control failing silently, which is the kind of bug that only shows up as a banned account — and we have now had two.
2. **Finish the sign-in flow.** Rotation issues a fresh account but Settings cannot yet sign into it; that currently requires starting a token search. Small, and it is the difference between a feature and a workaround.
3. **Prune retained raw profile data** and add a retention job that deletes candidate data past its useful life. This is the smallest piece of work with the largest compliance return.
4. **Connect an applicant tracking system.** With purchased data off the table this becomes the cheapest legitimate source available: candidates who already applied to us carry no legal ambiguity, and the same engine ranks them unchanged. It is the fastest way to put the tool to real work without a data budget.

Short term — weeks, and dependent on the source decision

4. **Make the AI layer economically sensible.** Move to a cheaper model tier, measure whether ranking quality changes, and cache results per candidate-and-role so the same person is not re-evaluated across similar searches.
5. **Improve title and location scoring.** Understand seniority as a level rather than a keyword; understand cities as places with regions and distances. These are the two most visible sources of a score that looks wrong to a recruiter.
6. **Let recruiters tune the weights per search.** The versioned scoring profile already makes this safe to expose — a role where location is non-negotiable and one where it is irrelevant should not be scored identically. This is a visible product feature sitting on top of work already done.
7. **Deploy it somewhere real**, with monitoring and alerting, so it can be used rather than demonstrated.

> **WHAT THE RESTRICTION CHANGES ABOUT THIS ORDER**
>
> Item 8 was already the recommendation. With the LinkedIn path now down and a second account lost, it stops being a planning question and becomes the thing blocking the product from being usable at all. Everything above it is maintenance on a supply we have just watched fail twice.

Strategic — the decision that unblocks everything else

8. **Settle the data source.** Every capacity, cost and legal question traces back to this one choice. Until it is made, the product is capped at roughly one recruiter and carries a risk nobody has formally accepted.
9. **Integrate with an applicant tracking system.** Greenhouse, Lever or Workday. Candidates already in the pipeline carry no legal ambiguity at all, and the same scoring engine ranks them unchanged. This may be the highest-value direction in the whole document: it turns a legally awkward sourcing tool into a clean internal one.
10. **Learn from recruiter behaviour.** Every save, every rejection and every hire is a labelled example. With enough of them the weights stop being a considered guess and start being fitted to what this company's

recruiters actually value — which is a defensible advantage no data vendor sells.

> **IF YOU TAKE ONE RECOMMENDATION INTO THE ROOM**
>
> Ask for a decision on the data source, and offer the ATS integration as the path that is both the cleanest legally and the most valuable commercially. It reframes the project from "a scraper with legal exposure" to "a matching engine that works on whatever data we legitimately have" — which is what was actually built.

18. Questions the owner will ask

Answers you can give without hedging. Short, honest, and leading somewhere useful.

"How much time does this actually save?"

The first-pass triage — reading and rejecting profiles that were never going to fit. For a role where a recruiter would open a hundred profiles, they now review a ranked shortlist and read the ones near the top. The collection itself is not instant on the scraping path — 20 profiles an hour — but it runs unattended while they do something else, and it is consistent in a way a tired human is not.

"Can we run this for the whole recruiting team tomorrow?"

Not on the scraping path. The 20-profiles-per-hour cap is per connected account, so five recruiters would share one budget — that is roughly one active user's worth of capacity. The product is built for more; the data supply is the constraint. Moving to a licensed source removes it. Purchasing data from a vendor would also have, but that option has been ruled out — so the realistic choice is licensing, or accepting the ceiling.

"Is this legal?"

The demo and purchased-data paths are clean. Scraping LinkedIn breaches their User Agreement — that is not a grey area. The realistic consequence is the account being permanently restricted, which has now happened twice — most recently to the account we were using, which is why that path is down right now. There is also a personal-data dimension: candidate profiles are regulated data, and we would need a documented lawful basis and a deletion process before real use at scale. I would not build the business on the scraping path, and the architecture was designed so we don't have to.

"I heard our account got banned. What happened?"

LinkedIn detected the automated access and restricted the account — the outcome I flagged as likely from the start, and the second time it has happened. The system handled it the way it was built to: it recognised the restriction, stopped the run, kept the profiles already collected, and locked the account out so nothing could touch it again. I have added an account-rotation path, so we can connect a different account and carry on today. But I want to be straight with you — that treats the symptom. It happened at 20 profiles an hour with every protection running, and the next account will probably go the same way. The fix is not a third account, it is a different data source, and that one is ruled out, so the honest answer is that we either license a source or accept that this serves one recruiter.

"Can't we just recover the banned account?"

Yes — LinkedIn's ID-verification checkpoint is the real route back and it usually works. Two catches. It permanently ties someone's government ID to that account, and if we then keep scraping with it, a second restriction on a verified account is far harder to come back from. So recovering it and continuing to use it for this are not compatible. My recommendation: if we want that account back for genuine professional use, verify it and never point this tool at it again.

"What happens if LinkedIn blocks us?"

Two different failures. If the *account* is restricted, that account is finished — it needs government ID to recover — and we would switch to a purchased source. If LinkedIn *redesigns their pages*, extraction returns empty profiles until it is re-tuned; the code writes a full page dump on failure specifically to make that an afternoon's work. In neither case does the rest of the product break, because the data source is a plug.

"Why should I trust the match score?"

Because you can check it. It is a weighted sum of four components — title 30, keywords 30, experience 30, location 10 — and every result stores its breakdown, so you can see exactly which component produced the number. It is deterministic: the same candidate against the same requirements always scores the same. And the rules are versioned, so a score from six months ago is still interpretable even though we have since changed the weights. There is no black box in the default configuration.

"Where's the AI? Everyone else says AI."

There is an AI layer and it is off by default, deliberately. The base scoring is rule-based because it has to be explainable and reproducible — those are requirements when you are ranking people. The AI layer solves one specific weakness: it recognises that "Backend Engineer" and "Backend Developer" are the same job, which keyword matching cannot. It blends evenly with the deterministic score rather than replacing it, so half of every number is always something we can show our working for. It costs one model call per candidate, which is why it is opt-in.

"Why isn't every candidate a good match?"

Usually the keyword wording. Keywords are matched against the candidate's headline and job titles, and they carry 30% of the score — so a term that only ever appears in someone's skills list scores zero. Use words that show up in job titles ("vendor management", "procurement") rather than tools. If a tool is genuinely non-negotiable, put it in Critical skills, which checks the skills list too. The system does handle word forms — "vendor management" matches "Vendor Manager" — so you do not need to guess exact phrasing.

"What does it cost to run?"

Infrastructure is tens of dollars a month — it is a small application. The real variable is data. Scraping is free in cash and expensive in risk. Purchased data is cents per profile, which makes cost a budget decision rather than a technical ceiling. AI matching adds one model call per candidate and should be moved to a cheaper model tier before being switched on. The line item people forget is engineering time to repair extraction when LinkedIn changes — a paid source moves that onto a vendor.

"How do I know it works? What's tested?"

358 automated tests, all passing, run against fixtures so they are fast and never flaky. The heaviest coverage is on the logic that produces the score — experience maths, skill matching, each scoring component, the filters — because that is where a silent error would produce a wrong answer that still looks plausible. What is not covered: live LinkedIn, which cannot be tested without scraping, and the frontend beyond type-checking.

"Why not just buy LinkedIn Recruiter?"

A fair question, and the answer is not "this is cheaper". LinkedIn Recruiter is the sanctioned tool and it has better data access than we will ever have. What it does not do is score candidates against *our* definition of a good fit, with weights we control, producing a ranked list with recorded reasons and a CSV. And it only works on LinkedIn — this engine will rank candidates from our own ATS, a purchased dataset, or any future source, using the same rules. The value is the matching layer, not the collection.

"What's left before we can actually use this?"

A decision on the data source, which gates everything. Then: deploy it somewhere real with monitoring, add the data retention and deletion job, turn authentication on properly, and fix two small quiet failure modes I have already identified. That is weeks, not months — the product itself is built and working.

"If you had to cut something, what would you cut?"

The scraping stack. It is the most technically interesting part and the least commercially defensible — it does not scale, it breaks on someone else's schedule, and it carries legal and account risk. Everything valuable sits above it and works identically with licensed data or with our own applicant records. I would rather spend that engineering effort on scoring quality and an ATS integration.

"What are you most worried about?"

That someone points the LinkedIn source at an account that matters. The controls are real — 20 per hour, manual sign-in, persistent budget, automatic lockout on restriction — but the failure is permanent and it has now happened twice. Whatever else we decide, the operating rule should be that only a dedicated, expendable account is ever connected. And I would rather we stopped needing that rule at all.

19. Glossary

Every technical term used in this document, in plain language. If the owner asks what something means, this is the answer that will not lose them.

TERM	MEANING
API	The part of the system other programs talk to. Here, the backend the web app sends requests to.
Worker	A separate program doing slow work in the background so the web app stays responsive.
Queue (Celery / Redis)	A list of jobs waiting to be done. The API adds jobs; workers take them off. Jobs survive a restart.
Provider	Our word for a candidate data source. All providers offer the same two operations, so they are interchangeable.
Scraping	Reading a website automatically, as a human would with a browser, and extracting the data from the page.
Playwright / Chromium	The software that drives a real browser under program control. Chromium is the browser itself.
Selector	The instruction that locates a piece of information on a page. Fragile selectors are why the earlier version broke.
Fingerprint	The set of characteristics a browser reveals — screen size, graphics card, timezone. Websites use them to recognise and link visitors.
Honeypot	An invisible element on a page that only automated software would click. Clicking one identifies you as a bot.
Rate limit	A cap on how often something may happen. Here: at most 20 profiles opened in any hour.
Rolling window	The last sixty minutes from right now, continuously — not a clock hour that resets on the hour.
Cache	Keeping a copy of something already fetched so it need not be fetched again. Here, profiles are reused for seven days.
Deterministic	The same inputs always produce the same output. The opposite of "it depends".
Score breakdown	The four component sub-scores stored alongside the total, so any score can be explained.
Score version	A label recording which set of rules produced a score, so old results stay interpretable after the rules change.
Hard filter	A pass/fail requirement. Failing one removes the candidate regardless of how well they score elsewhere.
Pre-filter	A cheap early check on shallow data, used to avoid opening profiles that are certainly wrong.
Normalisation	Reducing different forms of the same word to one, so "Vendors" and "vendor", or "manager" and "management", compare equal.
JWT / token	A signed proof of who you are, issued at sign-in and presented with each request.
Encryption at rest	Data stored scrambled, readable only with a key kept elsewhere. Applied here to browser session state.
Server-Sent Events (SSE)	A one-way live connection letting the server push updates to the browser — what drives the live progress display.

Migration	A versioned, repeatable change to the database structure, applied identically in every environment.
ORM	A layer letting the code work with database rows as ordinary objects, which also removes a whole class of security bugs.
Container / Docker	An application packaged with everything it needs, so it runs the same on any machine.
Fixture	Fixed sample data used for demonstrations and tests, so nothing depends on a live external service.
Actor (Apify)	A hosted scraper run by a vendor on their own infrastructure. We call their API; they do the collection.
ATS	Applicant Tracking System — the software a company already uses to manage applicants (Greenhouse, Lever, Workday).
Cascading delete	Removing a record automatically removes everything that depended on it. The basis of a workable deletion process.

Prepared from the codebase at commit `c9b0cb0` on branch `main`. All figures verified against the source; the test count was confirmed by executing the suite (358 passed). Legal statements summarise the project's own compliance documentation and are engineering analysis, not legal advice.